

Yume-4 ISA

Full Documentation

「夢を束ねて」

“Bundle up Your Dreams”

Darcy Stabler
Mar 3, 2026 – NAN
Version - 1.1

Overview

This document covers the Yume-4 Instruction Set Architecture (ISA) and how it works. This document also includes the Y-Float.01 (YF.01) specification used for floating point math.

Yume-4 is a 64-bit general purpose ISA designed to be used to run OSes and general applications.

— ISA Definition —

Base Spec:

GP Register size:	64-bit
GP Register file:	32 registers, R0 - R31. (R32 “exists” but is a constant 0)
ALU size:	64-bit / 32-bit
FPU size:	64-bit / 32-bit
Vector Register size:	512-bit
Vector Register file:	32 registers, SR0 - SR31
Vector ALU size:	64-bit - 8 lane / 32-bit - 16 lane
Vector FPU size:	64-bit - 8 lane / 32-bit - 16 lane
Opcode width:	8-bit standard
Parameter width:	8-bit * 3 parameters (64-bit immediate on some opcodes)
Memory addressing:	64-bit
Memory bus width internal:	512-bit
Memory bus width external:	256-bit
**MMIO bus width:	64-bit
Little-endian	
Weak memory ordering (*hardware dependent) (see: Memory Model)	
Y-Float FPU format (Yume-Float) (see: Y-Float)	
Floats use the same GP register file as integers	

*The Yume-4 ISA does not *guarantee* weak memory ordering, it is ultimately up to the hardware implementation whether weak ordering is actually implemented. All programs should assume [Y4WMO](#) unless targeting very specific hardware.

**MMIO is accessed the same as standard memory, starts at address 0xFFFF_FFFF_0000_0000, and has the same MMU protections as standard memory.

Register file

All registers in this table can be read from and written to in any mode with the exception of UPT

Register name	Symbolic name	Description
R0	A0 / RT	Argument val / return val
R1-7	A1- A7	Argument val
R8-15	S0 - S7	Saved registers
R16-28	T0 - T12	Temporaries
R29	RA	Return address
R30	TP	Thread pointer
R31	SP	Stack pointer
R32	C0	Constant zero
R33	FLG	Flag register
R34	CSR	Call Stack Register
R35	IID	Interrupt ID
R36	UPT	Uptime
R37	SPID	Scratchpad PID
R38	CSRD	CSR Depth

GP registers

- Every GP register is identical in hardware, with one exception for SP which is hard-wired to [PUSH](#) and [POP](#), but otherwise identical.
- GP registers are 64-bit and can be used by all [arithmetic operations](#) and most other operations.

C0

- C0 is not a real register, it is simply a hard-wired zero.
- Any attempt to write to C0 will result in a NOP

FLG

- FLG (flag register) is the register used to hold [flags](#).
- Each bit in the flag register corresponds to the equivalent N flag (bit 0 is F0)
- FLG is 32-bit. Anything written to it above 32-bits will truncate to 32-bits

CSR

- CSR (Call Stack Register) is a 64-bit register used to store return addresses, routine types (call & interrupt), and CPU state
- Bytes 0-7 at CSR form the return address, byte 8 is the type identifier, and byte 9 is the state byte
- Type 0x00 represents call, anything above represents interrupt, with the exact value representing the interrupt priority
- If a state flag bit is non-zero, the system will be set to enable the corresponding state/mode upon return, otherwise it will remain as-is. The current state is stored to the state byte on an interrupt entry
- Bytes are stored little endian 0-9 (B0 - 0x00, B9 - 0x09)
- CSR stores to system memory
- CSR stores then increments, loads then decrements

CPU state flags

0: user mode

1: Vector mode

2: kernel MMU enabled

IID

- IID (Interrupt ID) is a 64-bit register used to store the ID of the current [interrupt](#)
- IID is automatically set by the current interrupt

PC

- The Program Counter is a 64-bit register used to keep track of the current execution point in memory.
- The PC always points to either the instruction currently being loaded into memory, or the instruction about to be loaded into memory.
- The PC can be directly manipulated by control flow instructions

CSRD

- The CSR Depth register stores the depth of the Call Stack, storing 0 when there are no entries.
- CSRD increments upon any call or interrupt, and decrements upon any return.
- CSRD is not automatically zeroed upon manual CSR setting, and must be set by the OS.
- CSRD is automatically set to zero upon boot.

Instruction Set

Standard instruction:

- 8-bit opcode + 24-bit parameters (32-bit + 64-bit param extension if instruction uses it)

Instruction data layout:

Opcode, Std_Param, Extended_Param (EP)

XXXXXXXX_XXXX... XXXX

^8-bit_ ^24-bit_----

Flags

F0: Invalid Instruction

- Fault
- Triggered on any opcode that does not map to a known instruction
- Also triggered when any opcode's parameters are invalid, such as instructions attempting to access registers that do not exist

F1: div-zero

- Fault
- Triggered by any division variant with a zero as the divisor

F2: Data Segmentation Fault (SEG_D)

- Fault
- Triggered by any attempt to access unmapped or invalid memory in with a data load/store instruction

F3: Code Segmentation Fault (SEG_C)

- Fault
- Triggered by any attempt to execute unmapped or invalid memory

F4: Privilege Violation Fault (PrVi)

- Fault
- Triggered by any attempt to use a kernel mode instruction in user mode

F5: Scratchpad DMA active

- Triggered by either of the two scratchpad DMA instructions and disables when the DMA is finished

F6: Arithmetic overflow/underflow

- Triggered when any arithmetic operation underflows or overflows

F7: Invalid Interrupt

- Fault
- Triggered when an interrupt triggers with a vector of 0x00

F8: Scratchpad Segmentation Fault

- Fault
- Triggered when the scratchpad is accessed with a MMU base and SPID mismatch

Instructions:

32-bit loads and arithmetic operations zero extend the upper 32-bits of a register

*A, B, and C, refer to the ID of registers unless stated otherwise such as with "immediate value of..."

**Not representative of actual instruction size; purely to show order

Arithmetic

```

      C  B  A  Op
**0x00 00 00 00 //inCPU
**0x00 00 00 00 //inMEM
      Op A  B  C

```

Integer

0 ADD - 0 add

- Store the result of $B + C$ to A ($A = B + C$)
- 32-bit

1 SUB - 1 subtract

- Store the result of $B - C$ to A ($A = B - C$)
- 32-bit

2 MUL - 2 multiply

- Store the result of $B * C$ to A ($A = B * C$)
- Signed
- 32-bit

3 DIV - 3 divide

- Store the result of B / C to A ($A = B / C$)
- Signed
- 32-bit

4 MOD - 4 modulo

- Store the remainder of B / C to A ($A = B \% C$)
- Signed
- 32-bit

5 UMUL - 5 unsigned multiply

- Store the result of $B * C$ to A ($A = B * C$)
- Unsigned
- 32-bit

6 UDIV - 6 unsigned divide

- Store the result of B / C to A ($A = B / C$)
- Unsigned
- 32-bit

7 UMOD - 7 unsigned modulo

- Store the remainder of B / C to A ($A = B \% C$)
- Unsigned

- 32-bit

8 ADD64 - 8 add 64-bit

- Store the result of $B + C$ to A ($A = B + C$)
- 64-bit

9 ADD64I - 255 add immediate 64-bit

- Store the result of $B + EP$ to A ($A = B + EP$)
- 64-bit
- Extended Param

10 SUB64 - 9 subtract 64-bit

- Store the result of $B - C$ to A ($A = B - C$)
- 64-bit

11 MUL64 - 10 multiply 64-bit

- Store the result of $B * C$ to A ($A = B * C$)
- 64-bit
- Signed

12 MUL64I - 254 multiply immediate 64-bit

- Store the result of $B * EP$ to A ($A = B * EP$)
- 64-bit
- Signed
- Extended Param

13 DIV64 - 11 divide 64-bit

- Store the result of B / C to A ($A = B / C$)
- 64-bit
- signed

14 MOD64 - 12 modulo 64-bit

- Store the remainder of $B \% C$ to A ($A = B \% C$)
- 64-bit
- Signed

15 MOD64I - 253 modulo immediate 64-bit

- Store the remainder of $B \% EP$ to A ($A = B \% EP$)
- 64-bit
- Signed
- Extended Param

16 UMUL64 - 13 unsigned multiply 64-bit

- Store the result of $B * C$ to A ($A = B * C$)
- 64-bit
- unsigned

17 UMUL64I - 252 unsigned multiply immediate 64-bit

- Store the result of $B * EP$ to A ($A = B * EP$)
- 64-bit
- Unsigned
- Extended Param

18 UDIV64 - 14 unsigned division 64-bit

- Store the result of B / C to A ($A = B / C$)
- 64-bit
- unsigned

19 UMOD64 - 15 unsigned modulo 64-bit

- Store the remainder of B / C to A ($A = B \% C$)
- 64-bit
- Unsigned

20 UMOD64I - 251 unsigned modulo immediate 64-bit

- Store the remainder of B / EP to A ($A = B \% EP$)
- 64-bit
- Unsigned
- Extended Param

21 SHR - 16 bit shift right

- Bit shift right ($A = B \gg C$)($A = B / 2^C$)
- 64-bit

22 SHRI - 250 bit shift right immediate

- Bit shift right ($A = B \gg EP$)($A = B / 2^{EP}$)

- 64-bit
- Extended Param

23 ASHR - 17 arithmetic shift right

- Arithmetic shift right (preserve sign)
- 64-bit

24 SHL - 18 bit shift left

- Bit shift left ($A = B \ll C$)($A = B * 2^C$)
- 64-bit

25 SHLI - 249 bit shift left immediate

- Bit shift left ($A = B \ll EP$)($A = B * 2^{EP}$)
- 64-bit
- Extended Param

26 AND - 19 bitwise AND

- Store the result of B AND C to A ($A = B \text{ AND } C$)
- 64-bit

27 ANDI - 248 bitwise AND immediate

- Store the result of B AND EP to A ($A = B \text{ AND } EP$)
- 64-bit
- Extended Param

28 OR - 20 bitwise OR

- Store the result of B OR C to A ($A = B \text{ OR } C$)
- 64-bit

29 ORI - 247 bitwise OR immediate

- Store the result of B OR EP to A ($A = B \text{ OR } EP$)
- 64-bit
- Extended Param

30 XOR - 21 bitwise XOR

- Store the result of B XOR C to A ($A = B \text{ XOR } C$)
- 64-bit

31 NOT - 22 bitwise NOT

- Store the result of $\neg B$ to A ($A = \neg B$)
- C is unused and will be ignored regardless of values
- 64-bit

Float

32 ADDF - 23 add float

- Store the result of $B + C$ to A ($A = B + C$)
- YF32

33 SUBF - 24 subtract float

- Store the result of $B - C$ to A ($A = B - C$)
- YF32

34 MULF - 25 multiply float

- Store the result of $B * C$ to A ($A = B * C$)
- YF32

35 FMA - 26 Fused multiply add float

- Store the result of $B * C + (C+1)$ to A ($A = B * C + (C+1)$)
- C+1 refers to the register ID one above C
- YF32

36 DIVF - 27 divide float

- Store the result of B / C to A ($A = B / C$)
- YF32

37 ADDF64 - 28 add float 64-bit

- Store the result of $B + C$ to A ($A = B + C$)
- YF64

38 SUBF64 - 29 subtract float 64-bit

- Store the result of $B - C$ to A ($A = B - C$)
- YF64

39 MULF64 - 30 multiply float 64-bit

- Store the result of $B * C$ to A ($A = B * C$)
- YF64

40 FMA64 - 31 Fused multiply add float 64-bit

- Store the result of $B * C + (C+1)$ to A ($A = B * C + (C+1)$)
- C+1 refers to the register ID one above C
- YF64

41 DIVF64 - 32 divide float 64-bit

- Store the result of B / C to A ($A = B / C$)
- YF64

42 CMPF - 33 Compare float

- Store the comparison result of $B \geq C$ to A
- Result is stored as integer sign
- -1 = less than, 0 = equal, 1 = greater than
- YF32

43 CMPF64 - 34 Compare float 64-bit

- Store the comparison result of $B \geq C$ to A
- Result is stored as integer sign
- -1 = less than, 0 = equal, 1 = greater than
- YF64

44 FTI - 35 Float to 32-bit signed integer

- Convert float in B to integer and store to A
- YF32

45 ITF - 36 32-bit signed integer to float

- Convert integer in B to float and store to A
- YF32

46 FTI64 - 37 Float to 64-bit signed integer

- Convert float in B to integer and store to A
- YF64

47 ITF64 - 38 64-bit signed integer to float

- Convert integer in B to float and store to A
- YF64

48 ENVEC - 39 Enable Vector mode

- Switch ALU instructions to use vector registers and the vector ALU
- When enabled, all arithmetic instructions remap to vector units
- Immediate value instructions use the immediate value for all lanes

49 DVEC - 40 Disable Vector mode

- Switch ALU instructions to use GP registers and the GP ALU

Data Movement

```
      C  B  A  Op
**0x00 00 00 00 //inCPU
**0x00 00 00 00 //inMEM
      Op A  B  C
```

^Immediate value representing number of bytes. Only 1, 2, 4, 8 are valid.

50 LOAD - 246 Load value from memory

- Load a value into A using EP as an immediate address, and C as the size
- B is unused

- Extended Param

51 STORE - 245 Store value to memory

- Store a value from A using EP as an immediate address, and C as the size
- B is unused
- Extended Param

52 RLOAD - 41 Load value from memory

- Load a value into A using address stored in B, and C as the size

53 RSTORE - 42 Store value to memory

- Store a value from A using address stored in B, and C as the size

54 PCLD - 43 Load value from memory using PC offset

- Load a value into A using B as offset address, and C as the size
- Address is calculated as $(\text{signed})B + PC = (\text{unsigned})\text{addr}$

55 PCST - 44 Store value to memory using PC offset

- Store a value from A using B as offset address, and C as the size
- Address is calculated as $(\text{signed})B + PC = (\text{unsigned})\text{addr}$

56 MOV - 45 Move register

- Move B into A
- 64-bit

57 LOADI - 244 Load immediate

- Load EP to A
- 64-bit
- Extended Param

58 PUSH - 46 Push value to stack

- Push a value stored in A to the stack at [SP](#) using C as the size
- A is stored, then the SP is decremented by the size

59 POP - 47 Pop value from stack

- Pop a value into A from the stack at [SP](#) using C as the size
- The SP is incremented, then A is loaded

60 LOADV - 48 Load to Vector register

- Load a value to A using address stored in B
- C is unused

61 STOREV - 49 Store to Vector register

- Store a value from A using address stored in B
- C is unused

62 GPTV - 50 Move GP registers to Vector

- Move an array of 8 GP registers starting at B to Vector register A
- C is unused
- 64-bit

63 VTGP - 51 Move Vector register to GP registers

- Move Vector register B to an array of GP registers starting at A
- C is unused
- 64-bit

64 APCLD - 52 Load a value from memory atomically

- Load a value into A using B as offset address, and C as the size
- Address is calculated as $(\text{signed})B + PC = (\text{unsigned})\text{addr}$

65 APCST - 53 Store a value to memory atomically

- Load a value into A using B as offset address, and C as the size
- Address is calculated as $(\text{signed})B + PC = (\text{unsigned})\text{addr}$

66 CAS - 54 Compare and swap

- Atomically read value stored at B, compare to value stored in register C, write value C+1 to address B if values from address B and register C match, and store result of operation (0 fail, 1 succeed) to A
- C+1 refers to the register ID C+1 (C = R4, C+1 = R5)

67 SPLD - 55 Load value from scratchpad

- Load a value into A using address stored in B, and C as the size

68 SPST - 56 Store value to scratchpad

- Store a value from A using address stored in B, and C as the size

69 SPLDV - 57 Load a value from scratchpad to a Vector register

- Load a value to A using address stored in B
- C is unused

70 SPSTV - 58 Store a value to scratchpad from a Vector register

- Store a value from A using address stored in B
- C is unused

71 SPDLD - 243 Start DMA transfer to scratchpad from memory

- Move a region of memory from address B to the scratchpad at address A, using EP as the number of bytes (up to 512KiB)
- C is unused
- Scratchpad DMA flag is set while DMA is active
- Extended Param

72 SPDST - 242 Start DMA transfer from scratchpad to memory

- Move a region of the scratchpad at address B to memory at address A, using EP as the number of bytes (up to 512KiB)
- C is unused
- Scratchpad DMA flag is set while DMA is active
- Extended Param

73 FENCE - 59 Force memory operations to complete before using their values

- Parameter A dictates the type of fence. 0 represents a full fence; all memory operations must complete. 1 represents a release, 2 represents an acquire.
- Prevents context switch until memory operations are complete

-Data Movement Continued

Control Flow

```
      C  B  A  Op
**0x00 00 00 00 //inCPU
**0x00 00 00 00 //inMEM
      Op A  B  C
```

74 **CMP** - 60 compare signed integers

- Compare B and C and store result to A
- -1 = less than, 0 = equal, 1 = greater than
- 64-bit

75 JMP - 241 unconditional jump

- Jump to address stored in EP
- All 3 standard parameters slots are unused
- Extended Param

76 CJG - 240 compare jump greater than

- Jump to EP if A is greater than B
- Unsigned
- 64-bit
- Extended Param

77 CJE - 239 compare jump equal to

- Jump to EP if A is equal to B
- 64-bit
- Extended Param

78 CJL - 238 compare jump less than

- Jump to EP if A is less than B
- unsigned
- 64-bit
- Extended Param

79 CJNE - 237 compare jump not equal to

- Jump to EP if A is not equal to B
- 64-bit
- Extended Param

80 JPC - 61 PC relative jump

- Jump to address A with PC offset
- Address is calculated as $(\text{signed})A + PC = (\text{unsigned})\text{addr}$

81 JGPC - 62 PC relative jump greater than

- Jump to address C with PC offset if A is greater than B
- Address is calculated as $(\text{signed})C + PC = (\text{unsigned})\text{addr}$
- Unsigned
- 64-bit

82 JEPC - 63 PC relative jump equal to

- Jump to address C with PC offset if A is equal to B
- Address is calculated as $(\text{signed})C + PC = (\text{unsigned})\text{addr}$
- 64-bit

83 JLPC - 64 PC relative jump less than

- Jump to address C with PC offset if A is less than B
- Address is calculated as $(\text{signed})C + PC = (\text{unsigned})\text{addr}$
- Unsigned
- 64-bit

84 JNEPC - 65 PC relative jump not equal to

- Jump to address C with PC offset if A is not equal to B
- Address is calculated as $(\text{signed})C + PC = (\text{unsigned})\text{addr}$
- 64-bit

85 CALL - 236 call subroutine

- Jump to address EP and return using **RET**
- Return address is stored at CSR
- "Call" (0x00) is stored at CSR + 8
- CSR is incremented by 10 bytes
- Extended Param

86 CLPC - 66 call subroutine relative to PC

- Jump to address stored in A with PC offset, and return using **RET**
- Return address is stored at CSR
- "Call" (0x00) is stored at CSR + 8
- CSR is incremented by 10 bytes

87 INT - 235 trigger software [interrupt](#)

- Trigger software interrupt using EP as the interrupt ID
- Using an unset ID will trigger a fault interrupt with value INVINT (Invalid Interrupt)

- The immediate value of A is the priority (0 will trigger an invalid instruction error)
- Extended Param

88 RET - 67 return from call or interrupt (whichever is at CSR + 8)

- CSR is decremented by 10
- User mode is set if the user bit at CSR + 9 is true

-Control Flow Continued

Kernel Mode

```

      C  B  A  Op
**0x00 00 00 00 //inCPU
**0x00 00 00 00 //inMEM
      Op A  B  C

```

89 USR - 68 Set [user-mode](#) pending flag

- Enables the user-mode pending flag and set user mode on next control flow instruction
- User mode disables all kernel mode instructions and is disabled on interrupts

90 WFI - 69 wait for interrupt

- Halt execution until an interrupt is received
- Priority is set by the immediate value in A
- Interrupts below the set priority will not be received
- Priority temporarily overwrites the [Minimum Priority](#) register

91 TIMST - 70 set timer delay

- Set hardware timer delay in nanoseconds using value stored in A
- Timer interrupts hold maximum priority (255)

92 ENTIM - 71 toggle timer

- If hardware timer is not running, start running from the set value
- If hardware timer is running, stop running

93 KTLB - 72 "kill" table lookaside buffer (invalidate cache)

- Invalidate the MMU page cache for when the page table is updated
- The value stored in register A gives the virtual address to invalidate. If A stores 0, all TLB entries will be invalidated.

94 MMUST - 73 Set the [MMU](#) page base address

- The value in A sets the base physical address of the MMU page table, the immediate value of B sets kernel vs user mode (0 = kernel, 1 = user)
- C is unused

95 ENMMU - 74 enabled the [MMU](#)

- Toggle the MMU for kernel mode
- Does not affect user mode
- Only valid in the current context

-Kernel Mode Continued

Instruction encoding

To simplify decode complexity, instructions' opcodes do not directly align with the order in which they are defined under [Instruction Set](#). Non-EP (Extended Parameter) instructions take low opcodes starting from 0 moving up, while EP instructions take high opcodes starting from $(2^8)-1 = 255$ moving down. This ordering prevents EP and non-EP instructions from interfering with each other in future upgrades or extensions.

Non-EP Instructions:

0:	ADD	33:	CMPF
1:	SUB	34:	CMPF64
2:	MUL	35:	FTI
3:	DIV	36:	ITF
4:	MOD	37:	FTI64
5:	UMUL	38:	ITF64
6:	UDIV	39:	ENVEC
7:	UMOD	40:	DVEC
8:	ADD64	41:	RLOAD
9:	SUB64	42:	RSTORE
10:	MUL64	43:	PCLD
11:	DIV64	44:	PCST
12:	MOD64	45:	MOV
13:	UMUL64	46:	PUSH
14:	UDIV64	47:	POP
15:	UMOD64	48:	LOADV
16:	SHR	49:	STOREV
17:	ASHR	50:	GPTV
18:	SHL	51:	VTGP
19:	AND	52:	APCLD
20:	OR	53:	APCST
21:	XOR	54:	CAS
22:	NOT	55:	SPLD
23:	ADDF	56:	SPST
24:	SUBF	57:	SPLDV
25:	MULF	58:	SPSTV
26:	FMA	59:	FENCE
27:	DIVF	60:	CMP
28:	ADDF64	61:	JPC
29:	SUBF64	62:	JGPC
30:	MULF64	63:	JEPC
31:	FMA64	64:	JLPC
32:	DIVF64	65:	JNEPC

66:	CLPC	71:	ENTIM
67:	RET	72:	KTLB
68:	USR	73:	MMUST
69:	WFI	74:	ENMMU
70:	TIMST		

EP-Instructions:

255:	ADD64I	244:	LOADI
254:	MUL64I	243:	SPDLD
253:	MOD64I	242:	SPDST
252:	UMUL64I	241:	JMP
251:	UMOD64I	240:	CJG
250:	SHRI	239:	CJE
249:	SHLI	238:	CJL
248:	ANDI	237:	CJNE
247:	ORI	236:	CALL
246:	LOAD	235:	INT
245:	STORE		

-Instruction Encoding Continued

Scratchpad

Basic info

- **Max DMA transfer size:** 512KiB
- **Min DMA transfer size:** 1B
- **Addr range:** 0x0000 -> 0x0008_0000
- **Maximum access latency guarantee:** 2 core cycles
- **Bus width:** 512-bit
- **Byte aligned transfers**
- **Undefined at power-on**

The scratchpad is a 512KiB block of memory similar to L1 cache, but explicitly managed (this does not mean Yume-4 does not support a traditional cache hierarchy, it does.). Each core has its own separate scratchpad. The scratchpad uses a separate address space from system memory, so 0x00 in the scratchpad is not equal to 0x00 in system memory.

The scratchpad stores a 64-bit register alongside it that acts as the process ID enforcement, called the Scratchpad PID Register, preventing process A from accessing or corrupting process B's data. If the process ID register matches the current MMU base address, memory operations are permitted. If the process ID does not match, a Scratchpad Segmentation Fault is triggered and the OS kernel should handle stalling the user process until it can evict the old data and update the Scratchpad PID Register, or inform the user process that the scratchpad is not yet available.

When a [DMA transfer](#) in or out of the scratchpad is started, the [scratchpad DMA](#) flag will be set. When the DMA transfer completes, the scratchpad DMA flag will be unset and a scratchpad DMA complete interrupt (Hardware interrupt ID-2) will fire with priority 1 (minimum priority. 0 is reserved for call). If a DMA transfer is started while a DMA is already in progress, nothing will happen; the instruction will be ignored. (don't do it.)

Each core has its own scratchpad, meaning it is impossible for core-A to access core-B's scratchpad. Attempts to access outside the valid address range will cause a SEG_D fault

Interrupts

All interrupts increment the [CSR](#) by 10 bytes and add the appropriate entry. An interrupt with a vector of zero will trigger an invalid interrupt fault.

Interrupt registers

- **IPT base** - The IPT (Interrupt Priority Table) base registers are accessed via MMIO address 0x0000_0003. These registers store where in memory the IPT is. Register A accessed with the most significant bit of the address low is set for software interrupts, while register B accessed with the most significant bit set high is for hardware interrupts.
- **Fault Vector** - The Fault Vector register is accessed via MMIO address 0x0000_0004. This register stores the address to jump to upon a fault interrupt.
- **Hardware Vector** - The Hardware Vector register is accessed via MMIO address 0x0000_0005. This register stores the address to jump to upon a hardware interrupt.
- **Software Vector base** - The Software Vector base register is accessed via MMIO address 0x0000_0006. This register stores where in memory the SIT is.
- **Minimum Priority** - The Minimum Priority register is accessed via MMIO address 0x0000_0007. This register stores the minimum priority level for interrupts to be recognized. Setting this to 255 disables all interrupts (default behavior on boot)

Priority system

Interrupts work based on a priority system. When an interrupt line is triggered, the CPU checks that line's priority using the **Interrupt Priority Table (IPT)** and the current CSR type value. If the IPT value is greater than the current CSR type value and is greater than the Minimum Priority, an interrupt is triggered, otherwise the interrupt line is held high until the CSR or Minimum Priority is updated. Upon CSR update, if an interrupt line is high, it will check the CSR type value again.

Interrupt types

Fault

- Fault interrupts are triggered by any type of software fault.
- Fault interrupts automatically and immediately halt the current process, jump to the fault vector, and enable kernel mode (disable user mode).
- Fault interrupts have the maximum priority of 255.

Hardware

- Each hardware interrupt device has a unique ID which is passed through IID
- Hardware interrupts are triggered by any hardware interrupt line going high if the interrupt has high enough priority.
- When a hardware interrupt line goes high, it checks IPT base + ID for its priority

Software

- Each software interrupt type has a unique ID
- Software interrupts are triggered only by the INT instruction
- Software interrupts are stored as calls on the CSR
- When a software interrupt is called, it checks Software Vector base + 8(ID) for the address to jump to. It also checks its IPT base + ID for its priority.

State saving

On any interrupt, certain parts of the CPU state are saved, while others are not.

User mode is saved and restored by the hardware on interrupt entry and exit, along with Vector mode and the MMU mode, while everything else must be saved by the kernel or not at all.

(Calls do not save anything)

IDs

0 - Interrupt Timer

1 - keyboard

2 - scratchpad DMA complete

Hardware timer

Yume-4 has two hardware timers: the interrupt timer, and the uptime timer.

Interrupt timer

The interrupt timer is a 64-bit timer counting nanoseconds and set with [TIMST](#) and [ENTIM](#).

When the interrupt timer is enabled, it counts down from the set value. Once the timer reaches zero, it triggers a hardware interrupt with ID-0 and priority 255. Each core has its own interrupt timer

Uptime timer (UPT)

The uptime timer cannot be written to. It is only used to measure how many nanoseconds the system has been up for. The UPT cannot be accessed in user mode. All cores share one uptime timer

Privilege format

Kernel mode

- Kernel mode allows for all instructions to be available and disables [MMU](#) protections unless explicitly enabled.
- Kernel mode is the default mode on boot or on interrupts

User mode

- User mode restricts all kernel mode instructions and enables user [MMU](#) protections so unmapped memory cannot be accessed, read-only memory cannot be written to, and kernel-only memory cannot be accessed.

- User mode is temporarily disabled on any interrupt.

Transitioning

- To transition to user mode, the kernel must call [USR](#) to set the user-mode pending flag, then initiate a [control flow](#) change (ideally jumping to the user program).
- Upon reaching a control flow instruction, if the user-mode pending flag is set, the CPU will switch to user mode and enable the protections that come with it.

Boot behavior

- On boot all memory is UB, including scratchpad and non-architecture registers.
- The MMU is uninitialized on boot and all of its registers are UB.

Boot sequence

On boot, the CPU reads the bootloader from firmware to 0x0001_0000, then executes the bootloader, reading the first 16KiB of memory from the boot drive into memory at 0x0000 and jumping to that. From there, the kernel is responsible for loading any more of itself and initializing the rest of the hardware.

Thread model

- Each core has an ID 0 -> N that can be accessed through MMIO address 0x0000_0008, where the least significant byte is the current core ID and the second byte is the total number of cores.
- Threads can be initialized by sending an execution start point through MMIO address 0x0000_0008, where the most significant byte represents the core ID.
- Threads can be disabled by sending 0x00 through MMIO address 0x0000_0008, where the most significant byte represents the core ID.
- Atomic operations always commit in once cycle to prevent torn memory operations
- Atomic operations force memory to fall back to RAM or the lowest cache level that is shared by all cores.
- When initialized, the kernel should set up the thread's registers.

MMU

- Yume-4 uses a 3 level 47-bit MMU to map virtual addresses to physical memory.
- The Yume-4 MMU uses 16KiB pages.
- Once enabled, the MMU translates any address into virtual space. If the address is not mapped, it will trigger a fault interrupt.

Format

A 47-bit virtual address is split into sections—the top 3 being 11-bit and the bottom one being 14-bit—which are used to index into their respective tables to find the next table, and eventually the physical address. The MMU uses two separate page tables for kernel vs user mode. Page table entries are 8-bytes (64-bits) long.

Yume-4 MMU bit layout

47-bits: 0b000000000000_000000000000_000000000000_00000000000000

Use: ___ __^level-1____^level-2____^level-3____^offset

MMU output address bit layout

Header, **physical address**

64-bits: 0xFF_FF_FFFF_FFFF

Decode process

1. Given a 64-bit input address, the 17 most significant bits will be discarded, and the remaining 47 will be split into 3 11-bit sections and one 14-bit section.
2. Section 1 (most significant 11) will be used to index into the top level page table. The 8-byte value at the index will be used as a pointer to the second level page table.
3. Section 2 will be used to index into the second level page table the same as section 1.
4. Section 3 is used to index into the third level page table, which then gives a base address which is added to the offset to give the final physical address.
5. If section 2 has the leaf flag enabled, the remaining bits will be used to make a larger 32MiB page.

Flags

Format: 0bXXXX_(rest of physical address)

- Bit 0: Valid – Is this page valid? If not trigger fault (LSB)
- Bit 1: Read-Only – If write operation, trigger fault
- Bit 2: Executable – If not true and PC touches this, trigger fault
- Bit 3: Leaf – Leaf node; table index + remaining bits = physical address (MSB)

MMIO bus

Standard protocol

- Upon sending a chunk of data over the MMIO bus, the hardware will increment an 8-bit “header” to the target address (overflow will simply reset back to 0 and continue).
- MMIO devices should also update their 8-bit header to the CPU to signal new data
- Checking for new data involves reading the header and comparing it to the last known value. If it differs, new data has been sent.

Header, Data

Format: 0xFF, 0xFFFF_FFFF_FFFF_FFFF

Addressing

- Addressing an MMIO device on Yume-4 means using a data read or write operation at that device’s ID + 0xFFFF_FFFF_0000_0000

MMIO devices

This section includes interrupt IDs of certain devices

TTY Handler

0x0000_0000

Input

8-byte character input – receive sent characters

1 char / byte, byte 0-7

Output

1-byte ready output – send the ready signal

Output !0 = ready for new input

Behavior

The TTY device sets its output port to non-zero to indicate that it’s ready for a new packet of up to 8 characters. Upon receiving a packet of characters, the TTY will set its ready byte low and increment the header byte, print the characters, then set its ready byte high and increment the header byte again.

Keyboard Handler

0x0000_0001

Input

1-byte CPU ready – check if CPU is ready for a new key

Input !0 = ready for new key

Output

1-byte output – ASCII character

Output the current pressed ASCII character

Behavior

The keyboard handler polls the keyboard for key inputs approximately every 1ms, storing the char values in a ring buffer. Every cycle, if any keys are in the buffer, the keyboard checks the CPU ready port. If the CPU ready port is !0—indicating the CPU is ready to handle a new key—it sets the output port to the key value, then fires a hardware interrupt with ID-1. When the CPU reads a key, it should acknowledge it by setting its ready byte high. (the ready byte does not have to be set low beforehand because the keyboard handler will check its header for an update)

PowerState Handler

0x0000_0002

Input

1-byte power mode – update power profile

0: P2 – low power (default). Clocks significantly reduced

1: P1 – reduced power. Clocks slightly reduced

2: P0 – maximum power. Clocks set to default maximum

3: PN1 – 0C. Clocks set past default maximum if possible

4: Shutdown

5: Reboot. Triggers shutdown then sends power on signal

Behavior

Setting a mode flag will immediately trigger the specified P-state. P-states are not saved between shutdowns or reboots. Upon boot, P-state is set to P2

Interrupt Setup Handler

0x0000_0003 -> 0x0000_0007*

*each core points to its own private device

Input

0x0003 -> 8-byte IPT base – Interrupt Priority Table base address

0x0004 -> 8-byte Fault Vector – Fault interrupt address

0x0005 -> 8-byte Hardware Vector – Hardware interrupt address

0x0006 -> 8-byte Software Vector base – Software Interrupt Table (SIT) base address

0x0007 -> 1-byte Minimum Priority – Minimum priority for interrupts to trigger

Behavior

Setting any input address will automatically set that register of the Interrupt Setup Handler. These values are all uninitialized on boot with the exception of Minimum Priority, which defaults to the maximum of 255 (disabling interrupts).

The most significant bit of IPT base switches between hardware and software interrupts. If the most significant bit is high, the IPT base will be set for hardware interrupts, otherwise it will be set for software interrupts.

Thread State Handler

0x0000_0008*

*each core points to its own private device

Input

8-byte core setup – Core ID and start point

The most significant byte is the core ID, the bottom 56 bytes are the core's execution start point.

Output

2-byte core ID – Current core ID and total number of cores

The most significant byte is the total number of cores on the system, the least significant byte is the current core's ID

Behavior

When a new value is received, the Thread State Handler initializes the core in the ID field, then sets its execution start point to the start point field. If the start point field is all zero, the core is

disabled. When a core polls the core ID output, it is sent its ID and the total number of cores on the system.

Memory Model – Y4WMO

Yume-4 Weak Memory Ordering

1 – Overview

Yume-4 uses a weak memory model, meaning memory operations can be reordered internally to allow for more efficient computation. The following portion of this document details how this memory model must behave, what is guaranteed, and what isn't guaranteed.

All Yume-4 memory operations are either a load or a store, or both. For example, **PUSH**, **STORES**, **APCST**, are all store operations, while their counterparts are all load operations. Some operations, namely **CAS**, are both a load and a store.

2 – Atomicity

2.1 – Overview

Atomicity for Yume-4 is defined as a load or store instruction in which the entire portion of data is committed in one clock cycle, preventing torn reads and writes. (note that while atomic operations must be committed in one clock cycle, they are not required to execute in one clock cycle.)

2.2 – Requirements

In order for a load or store operation to be atomic, it must be an atomic operation such as **APCLD**, and it must not cross a page boundary.

If an atomic operation crosses a page boundary, it will still load or store a value, but it cannot be guaranteed to not tear at the boundary, and may take multiple cycles to commit, breaking the atomicity guarantee for that operation. (programmers and compilers should avoid atomic operations across page boundaries if at all possible.)

Naturally aligned load and stores are automatically considered atomic, so long as they do not cross a page boundary. Naturally aligned means the base address is cleanly divisible by the size of the piece of data.

Atomic operations are supported for any power of two widths between 1-8 bytes.

3 – Single Core Order

3.1 – Overview

Single Core Order describes the ordering guarantees that the processor provides to a single program within a single processor core.

3.2 – Order Guarantees

In Yume-4, within a single context on a single core, memory operations are allowed to be re-ordered and delayed internally, but they **MUST** appear in order to the program. If the program stores a value, then loads said value, it must load the most recently stored value to that address. If a program stores A, then stores B, any loads from the program at that address must load B with no exceptions.

If a 4-byte value is stored to 0x00, then a 4-byte value is read from 0x03, it will load the last byte of the store, along with the most recent data stored to 0x04-0x06.

3.3 – External Guarantees

While memory operations are allowed to appear out of order to other cores or contexts, they still have restrictions to ensure security and stability.

Memory operations must never be visible before a branch that determines if said memory operation happens. This means that if the processor predicts branch A, it can execute and build a speculative store buffer for said branch, but it cannot commit any values to memory until it is determined with 100% certainty that branch A will be taken. Memory load operations, however, can be performed ahead of time on uncertain branches, so long as they are not MMIO loads, which must wait for the branch to be 100% certain.

4 – Inter-core synchronization

4.1 – Overview

Inter-core synchronization describes the way in which cores ensure data is synchronized between each other, rather than just within a single core.

4.2 – FENCE

4.2.1 – FENCE Overview

FENCE in Yume-4 is by default a standard full FENCE operation. Unlike many ISAs, Yume-4 does not have explicit acquire/release instructions—instead FENCE takes the value of parameter A to determine a full fence, an acquire, or a release.

4.2.2 – FENCE fence

If FENCE has a parameter A value of zero, it is just a standard full fence. This is the simplest, but slowest mode of FENCE. A full fence forces all memory operations in the memory operation buffer to be committed before it allows any new memory operations to be executed. A full fence does not explicitly prevent execution of code; code that does not rely on any new loads or stores is not halted by a full fence. However, if the code has any memory operations after the full fence, it will be halted at the first memory operation until the buffer is fully flushed.

4.2.3 – FENCE release

If FENCE has a parameter A value of 1, it becomes a release operation. A release operation requires all store operations to commit before any memory operations after the release are allowed to execute. The mechanics are similar to a full fence except that it only prevents memory operations from being moved from behind to in front of it in execution. The typical pattern would be to perform a memory operation, then use a release to ensure the memory operation is committed immediately (relative to anything in front of it).

4.2.4 – FENCE acquire

If FENCE has a parameter A value of 2, it becomes an acquire operation. An acquire operation requires all load operations to execute only after the acquire. The mechanics of an acquire differ slightly from a full fence or release in that they don't prevent anything from being moved to in front of the acquire. Acquire only prevents load operations from being moved to an earlier state in the code relative to the acquire. The typical pattern for an acquire would be to use an acquire, then perform a memory operation to ensure the memory operation is forced to load up-to-date information.

4.3 – CAS (Compare and Swap)

4.3.1 – CAS overview

A Compare and Swap operation in Yume-4 describes atomically reading a value, comparing it to another value, and writing yet another value if the first two values match. CAS also stores the result (fail / succeed) to a register.

4.3.2 – CAS operation

When a CAS operation is used, the address stored in the register B (B from parameter B) is loaded atomically and compared to the value stored in register C. If the value loaded and the value stored in register C match, the value stored in C+1 (defined as the ID of C + 1, so R7 becomes R8) is written to the address stored in register B.

CAS is unique in that it is the only atomic memory operation in Yume-4 that may take multiple cycles to complete and still be atomic. If an implementation requires multiple cycles for CAS to commit, it must lock with the memory controller the address used by CAS, from the point of the load to the point of the store or failure. If another core or thread attempts to access the locked memory location of CAS during its operation, the memory controller **MUST** queue its access and allow it to continue after the CAS finishes; it must never allow the memory operation to fail.

5 – Scratchpad

The scratchpad has a relatively simple memory model compared to the system memory, so much so that I didn't bother giving it subsections.

Memory operations to the scratchpad are allowed to be re-ordered, but they follow the same requirements as Single Core Order. As the scratchpad is not accessible from any other cores, there are no synchronization instructions for it.

If a scratchpad DMA operation is triggered, the only ordering guarantees it has is that any operations committed before it starts will be visible, and that anything after it finishes will see the updated data. The DMA operation is not finished until the DMA-finished flag is set high. The memory state of the DMA region during a scratchpad DMA is undefined behavior.